Complemento Práctico: Diseño Modular

Del Caos al Orden y los Secretos del Javadoc

Mientras el PDF de la cátedra define qué es un método y su sintaxis formal, este material se enfoca en la **estrategia práctica**. ¿Por qué modularizar representa un verdadero punto de inflexión en nuestra carrera como programadores? La respuesta va más allá de la simple organización: se trata de pensar como profesionales del software.

En estas notas exploraremos tres aspectos fundamentales que complementan la teoría tradicional. Primero, comprenderemos la filosofía detrás de la modularización y por qué el código monolítico es el enemigo silencioso de todo proyecto. Segundo, revelaremos una configuración oculta en NetBeans que permite generar documentación web incluso para métodos privados, algo vital para el estudio y la auditoría interna del código que raramente se menciona en los manuales básicos. Finalmente, aplicaremos la regla de la Responsabilidad Única para transformar código caótico en soluciones elegantes y mantenibles.

Contenido

Un Antes y un Después: La Evolución del Código
Explora la transformación de código monolítico a modular.

Javadoc Avanzado: La Limitación por Defecto
Entiende las restricciones estándar de Javadoc para la documentación.

El Truco de NetBeans: Documentación Completa
Descubre cómo configurar NetBeans para una documentación exhaustiva.

Comparativa de Código: Espagueti vs Modular
Visualiza las diferencias entre código caótico y modular.

La Regla de Oro: Responsabilidad Única
Aprende el principio fundamental para un diseño de software limpio.

Beneficios Cuantificables de la Modularización
Conoce las ventajas tangibles de aplicar el diseño modular.

Resumen y Recomendaciones Finales
Conclusiones clave y consejos prácticos para tu desarrollo.

Un Antes y un Después: La Evolución del Código

Todos comenzamos de la misma manera: escribiendo todo el código dentro del método main. Es natural, es intuitivo, y funciona... hasta que deja de funcionar. Imaginemos un programa de 300 líneas, todas secuenciales, todas en un único bloque. Cuando algo falla, tenemos que leer línea por línea, perdidos en un mar de variables y condicionales. Es como intentar encontrar una aguja en un pajar cuando ni siquiera sabemos en qué parte del pajar buscar.

El Problema del Código Monolítico

Cientos de líneas en main, difícil de mantener, imposible de reutilizar, propenso a errores y frustrante para depurar

La Solución: Divide y Vencerás

Dar nombre a las acciones, crear abstracciones, transformar guiones técnicos en índices legibles por humanos

Pero modularizar no es simplemente "copiar y pegar código en otro lado". Es una transformación conceptual profunda. Es **dar nombre a las acciones** y crear niveles de abstracción que permitan pensar el problema en términos más humanos. En lugar de tener 20 líneas de operaciones matemáticas complejas, tenemos una llamada elegante: `calcularPromedio()`. Nuestro main deja de ser un guion técnico ininteligible y se convierte en un índice de acciones legible, donde cada línea cuenta una historia clara de lo que hace el programa.

> *"El buen código es como una buena prosa: cada función debe hacer una cosa, hacerla bien, y tener un nombre que describa exactamente qué hace."*

Javadoc Avanzado: La Limitación por Defecto

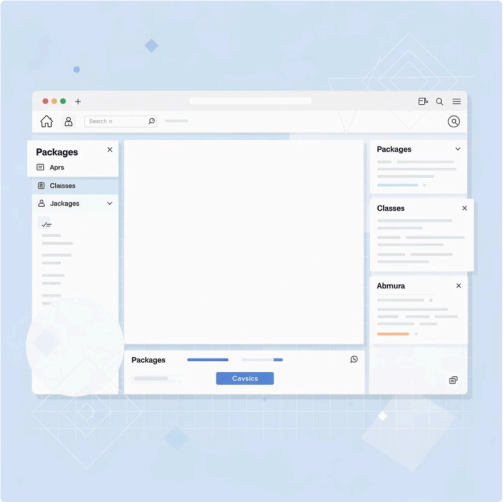
El apunte de cátedra enseña meticulosamente a escribir los comentarios de documentación con la sintaxis `/** ... */`. Aprendemos las etiquetas `@param`, `@return`, y `@throws`. Todo parece perfecto hasta que intentamos generar la documentación oficial.

Cuando navegamos a **Run → Generate Javadoc** en NetBeans, el IDE trabaja durante unos segundos y genera una hermosa página web con estilos profesionales y navegación estructurada. Pero al abrirla, nos encontramos con una sorpresa decepcionante: ¡está prácticamente vacía! ¿Dónde están todos nuestros métodos cuidadosamente documentados?

La respuesta está en la filosofía de diseño de Java. La herramienta estándar de generación de Javadoc **solo documenta lo público**, es decir, la API pública de nuestras clases. Oculta deliberadamente los métodos private o de paquete porque asume que son secretos internos de implementación que no deben exponerse al mundo exterior.

El Problema Académico

Como estudiantes, necesitamos ver **todo** documentado para poder estudiar nuestra propia lógica interna, no solo lo que exponemos públicamente. Esta configuración por defecto dificulta el aprendizaje.



El Truco de NetBeans: Documentación Completa

01

Acceder a Propiedades del Proyecto

Hacer clic derecho sobre el proyecto (el icono de taza de café en la vista de proyectos) y seleccionar **Properties** en el menú contextual

02

Navegar a la Sección Documenting

En la lista de categorías del panel izquierdo, buscar y seleccionar la opción **Documenting** para acceder a la configuración de Javadoc

03

Activar Documentación Privada

Buscar y marcar la casilla que dice: **"Include Private and Package Private Members"**. Esta es la opción clave que desbloquea todo

04

Aplicar y Generar

Hacer clic en OK para guardar los cambios. Ahora, al volver a ejecutar Generate Javadoc, verán todos sus métodos auxiliares documentados

Esta simple configuración transforma completamente la utilidad del Javadoc para propósitos académicos. Ahora podrán revisar toda su lógica interna de forma visual y profesional, facilitando tanto el estudio personal como la preparación de entregas. Es especialmente útil cuando trabajamos en proyectos complejos donde necesitamos recordar qué hace cada método auxiliar semanas después de haberlo escrito.

Nota importante: Esta configuración es por proyecto. Si crean un nuevo proyecto, deberán repetir estos pasos. Consideren crear una plantilla de proyecto con esta configuración ya activada para futuros trabajos prácticos.

Comparativa de Código: Espagueti vs Modular

La mejor forma de comprender el valor de la modularización es verla en acción. Analicemos dos implementaciones de la misma funcionalidad: calcular y mostrar el promedio de dos números ingresados por el usuario. La diferencia no está en lo que hace el código, sino en **cómo lo hace**.

Código Espagueti (Difícil de leer)

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Ingrese a: ");
    int a = Integer.parseInt(sc.nextLine());
    // Parseo repetitivo

    System.out.print("Ingrese b: ");
    int b = Integer.parseInt(sc.nextLine());
    // Parseo repetitivo

    int r = (a + b) / 2; // Lógica de negocio mezclada
    System.out.println("Promedio: " + r);

    // Si queremos repetir esto, tenemos que copiar y pegar todo el bloque
}
```

Problemas: Responsabilidades mezcladas, difícil de probar, imposible de reutilizar, variables crípticas (a, b, r), la limpieza del buffer (Integer.parseInt) hace que la lectura sea verbosa y ensucie la vista principal.

Código Modular (Profesional)

```
public class EjemploModular {

    // 1. Declaramos el Scanner a nivel de clase para que todos los métodos lo
compartan
    static Scanner sc = new Scanner(System.in);

    // 2. El main se convierte en un orquestador legible
    public static void main(String[] args) {
        int nota1 = leerEntero("Ingrese nota 1: ");
        int nota2 = leerEntero("Ingrese nota 2: ");

        mostrarResultado( calcularPromedio(nota1, nota2, 2) );
    }

    // 3. Método encapsulado: Esconde la complejidad de la consola y el parseo
    private static int leerEntero(String mensaje) {
        System.out.print(mensaje);
        return Integer.parseInt(sc.nextLine());
    }

    // 4. Método de negocio: Se encarga SOLO de la matemática (Responsabilidad
Única)
    private static int calcularPromedio(int a, int b, int cantidadNum) {
        return (a + b) / cantidadNum;
    }

    // 5. Método de salida: Se encarga SOLO de imprimir
    private static void mostrarResultado(int resultado) {
        System.out.println("Promedio: " + resultado);
    }
}
```

Ventajas: Responsabilidades separadas, fácil de probar, totalmente reutilizable, nombres descriptivos. El método leerEntero() esconde la lógica compleja de lectura y parseo.

> El main modular se lee como un libro: 'Leer primera nota, leer segunda nota, mostrar el resultado del promedio'. No necesitas ser programador para entender qué hace.

La Regla de Oro: Responsabilidad Única

Un Método, Una Responsabilidad

Cada función debe hacer exactamente una cosa y hacerla bien. Si un método calcula Y muestra Y valida, está haciendo demasiado.

Reutilización sin Duplicación

El código modular se invoca, no se copia. Si encuentras código duplicado, necesitas un nuevo método.

Facilidad para Probar

Funciones pequeñas y específicas son infinitamente más fáciles de probar y depurar que bloques monolíticos.

El Principio de Responsabilidad Única (Single Responsibility Principle) es uno de los pilares del diseño de software profesional. Aplicado a métodos, significa que cada función debe tener una razón, y solo una, para cambiar. Si estamos escribiendo un método que valida datos Y los procesa Y los guarda, entonces ese método tiene tres razones para cambiar en el futuro, lo que lo hace frágil y difícil de mantener.

1

Identificar

¿Qué hace este bloque de código?

2

Nombrar

Darle un nombre descriptivo

3

Extraer

Convertirlo en método independiente

4

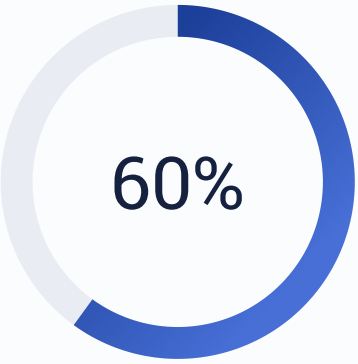
Reutilizar

Invocarlo donde sea necesario

Tip práctico: Si tienen que explicarle a alguien qué hace un bloque de código en particular, ese bloque probablemente debería ser un método con ese nombre exacto como su identificador. Por ejemplo, si dicen "este código verifica que el email sea válido", ese código debería estar en un método llamado `verificarEmailValido()`.

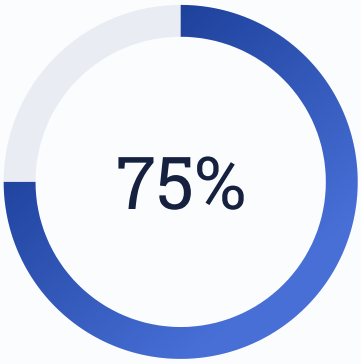
Beneficios Cuantificables de la Modularización

Modularizar no es solo una cuestión estética o de "buenas prácticas" abstractas. Tiene impacto medible y cuantificable en la calidad del software y la productividad del equipo de desarrollo. Diversos estudios en ingeniería de software demuestran que el código bien modularizado reduce dramáticamente el tiempo de mantenimiento y la tasa de defectos.



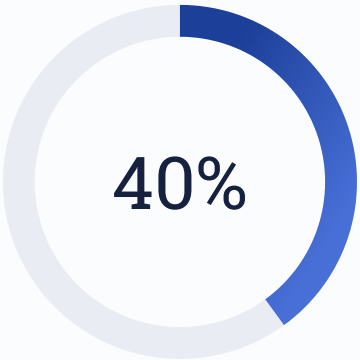
Reducción en tiempo de depuración

Los bugs se localizan y corrigen más rápido en código modular



Menos líneas duplicadas

La reutilización elimina redundancia y reduce superficie de error



Menor tiempo de onboarding

Nuevos desarrolladores comprenden código modular más rápidamente

Ventajas a Largo Plazo

- **Mantenibilidad:** Cambios localizados sin efectos colaterales en todo el sistema
- **Escalabilidad:** Agregar funcionalidad nueva es más simple y seguro
- **Colaboración:** Múltiples desarrolladores pueden trabajar en paralelo sin conflictos
- **Documentación:** El código se autodocumenta con nombres de métodos descriptivos
- **Testing:** Pruebas unitarias específicas para cada unidad funcional

Inversión Inicial vs Retorno

Sí, modularizar requiere más escritura inicial: definir métodos, especificar firmas, documentar parámetros y retornos. Es tiempo invertido en diseño antes de la implementación.

Pero ese tiempo se recupera multiplicado en cada fase posterior: depuración, mantenimiento, ampliación de funcionalidad, y trabajo en equipo. Es como construir con buenos cimientos: lleva más tiempo al principio, pero el edificio dura décadas.

Resumen y Recomendaciones Finales

Hemos recorrido el camino desde el código monolítico hasta el diseño modular profesional, explorando no solo el *qué* y el *cómo*, sino especialmente el *por qué*. La modularización no es una moda pasajera ni una complicación innecesaria: es la diferencia entre código que funciona hoy y código que seguirá funcionando mañana.

1

Principio de Nomenclatura Clara

Si tienen que explicar qué hace un bloque de código, ese bloque debería ser un método con ese nombre exacto. El código debe ser autoexplicativo.

2

Uso Estratégico de Javadoc

Activen la opción "Include Private" en NetBeans y generen documentación completa antes de entregar trabajos prácticos. Es su mejor herramienta de revisión.

3

Refactorización Continua

No esperen a que el código sea un caos para modularizar. Refactoricen constantemente: cuando un método supera 20 líneas, pregúntense si puede dividirse.

Checklist de Autoevaluación

Antes de entregar cualquier trabajo práctico, verifiquen:

- ¿Mi main tiene menos de 30 líneas?
- ¿Cada método tiene una única responsabilidad clara?
- ¿Los nombres de métodos describen exactamente qué hacen?
- ¿Hay código duplicado que podría extraerse a un método común?
- ¿La documentación Javadoc está completa incluyendo métodos privados?
- ¿Puedo explicar el flujo del programa leyendo solo los nombres de métodos?

Si respondieron "sí" a todas estas preguntas, están en el camino correcto hacia la programación profesional.

Recursos Complementarios

Consulten el PDF de cátedra para sintaxis formal de métodos y el video para ejemplos prácticos de refactorización en vivo.

> **"El código se escribe una vez, pero se lee cien veces. Escriban para los humanos, no para las máquinas."**